TopEndLibraryHub

Group Portfolio

HIT339 Distributed Development | Group 8

16 October 2026

Group member	Student number
Kamalpreet	S389008
Harsimran Singh	S388232
Mehakpreet Kaur	S390303
Sandesh Bhattarai	S390535

A family can keep its place while an item is repaired, moved to the chosen branch and made ready for collection.

Source repository | Project board

Windows-tested revision: **b79faf3**. Source branch: integration/group8-release-16oct.

Recorded results: 33 Windows tests and ten HTTPS API checks passed; actual imports and report/payment reconciliation verified. The separate VM installation result is recorded in section 8.

Contents

1. A connected library service 3

2. Our agreement, approach and plan 4

3. Requirements and decisions 9

4. Discussion minutes 13

5. Integration and user journeys 16

6. Individual work, hours and reflections 18

6.1 Kamalpreet - S389008 18

6.2 Harsimran Singh - S388232 21

6.3 Mehakpreet Kaur - S390303 23

6.4 Sandesh Bhattarai - S390535 25

7. UX feedback and refinements 28

8. Installation and running guide 29

9. Patron search and self-service guide 32

10. Verification and source quality 34

11. Selected evidence 37

12. Evaluation, sources and acknowledgement 42

1. A connected library service

We expanded Kamalpreet's Assignment 2 application into seven access channels and operational modules. Our main example addresses a practical problem: a family should keep its place in a waiting list while a toy is repaired or moved to the branch where it can be collected.

A repaired copy at Darwin is not yet ready at Palmerston. A physical Available copy can still be promised to another patron. If the catalogue, kiosk and report disagree, staff and patrons receive different answers. We connected the queue, item condition, physical location, checkout and simulated messages so that those stages agree.

Addition	Practical value	Example checked
Kiosk/private account	Scan and borrow, or follow only the current member's requests.	Identity-linked checkout, inactive/unlinked guards and Sky's account.
Secure API	Another application can read catalogue projections without write/account access.	Ten HTTPS checks for keys, endpoints, validation and rejected POST.
Simulated messages	Explain events and avoid repeated channel records.	No early pickup notice; Email/SMS after destination arrival.
Branches/desks/transfers	Trace origin, destination, dispatch and receipt.	Wrong-branch receipt and transit checkout rejected.
Holds/queue	Keep a fair request through return, repair and movement.	Asha/Noah order, cancellation and next-member promotion.
Importer	Add physical copies in bulk with validation feedback.	Two rows create three copies; repeated upload creates none.
Manager exports	Explain borrowing, fines and inventory outside the dashboard.	Matching CSV/PDF and a non-zero recorded payment.

The value we demonstrate is consistency across this journey: patrons retain their queue position and staff can explain when and where collection is possible. We retained one MVC deployment and database to support the horizontal expansion required by the brief.

2. Our agreement, approach and plan

HIT339 Distributed Development | Group 8 | TopEndLibraryHub | 16 October 2026

Our agreement

We selected Kamalpreet's Assignment 2 application as our common baseline. All four of us accepted the original agreement on the day it was created and shared. The calendar date of acceptance was not retained.

We each take responsibility for implementing, explaining, testing and handing over our assigned modules. We work on feature branches and coordinate shared models, migrations, Identity and circulation changes. We review shared files during conflict resolution and test the combined application after integration. We communicate availability, blockers and handoffs through our discussions and follow-ups. Kamalpreet coordinates dependencies, the release and portfolio.

We use feature branches, pull requests and a shared task register to organise the work. Each member provides a contribution, reflection and time record. Estimated hours are labelled, and private configuration stays outside the repository.

Member	Student number	Agreed responsibility
Kamalpreet	S389008	Lead; holds, simulated notifications, circulation integration, acceptance and portfolio.
Harsimran Singh	S388232	Branches, desks, staff location, transfers, installation guide and VM check.
Mehakpreet Kaur	S390303	Kiosk, secure catalogue API, patron guide and feedback.
Sandesh Bhattarai	S390535	CSV importer, manager analytics/exports and component tests/handoff.

Our plan and actual progress

Period	Planned work	Actual progress
3-4 October	Baseline, repository, board, roles and requirements.	We established the common project and allocations.
10-12 October	Modules and shared dependencies.	Identity/Borrower, pickup branch and InTransit=5 coordinated; branch foundation preceded transfers.
13-14 October	Integration and cross-module tests.	Branch evidence and hold, repair, transfer, collection/private-patron checks retained.
15 October	Internal completion and handoffs.	Reports received; kiosk/API reached master. Import/report fixes continued later; VM/integration still open.
16 October	One verified release and portfolio.	Combined source, 33 Windows tests, ten API checks, actual imports, report reconciliation and AUD 27 payment transition completed. Portfolio compiled; VM remains blocked.

Our official deadline is 16 October 2026, 23:59 Darwin.

Repository: https://github.com/Kamal03102006/HIT339_Group8_TopEndLibraryHub

Project board: https://github.com/users/Kamal03102006/projects/1

Both pages were publicly readable without authentication in the 16 October access check. Our final board review at 22:20 Darwin verified 17 assigned tasks: 14 Done, two In Progress and one Todo. The four foundation checks, seven modules, combined acceptance, guides and portfolio were marked Done. The separate VM check remained blocked. The delivery register below links the completed work to its evidence.

Task completion register

Status from the public project-board review on 16 October 2026 at 22:20 Darwin. The links identify the corresponding repository issues.

Task	Owner	Status	Deliverable or result
#1 Roles and team agreement	All four; Kamalpreet coordinates	Done	Four agreed responsibilities and individual handoffs.
#2 Requirements and acceptance	Kamalpreet	Done	Seven modules, acceptance checks and shared business rules.
#3 Clone, build and run	All four	Done	All four members confirmed local clone/build/run; board issue #3 is Done.
#4 Shared data and migration contract	Kamalpreet and Harsimran	Done	Branch/Identity links, InTransit=5, seven canonical migrations and SQL checks.
#10 Kiosk and private account	Mehakpreet	Done	Identity-linked checkout, private account and session controls; PR #8 and PH evidence.

Task	Owner	Status	Deliverable or result
#11 Secure catalogue API	Mehakpreet	Done	Key-protected read-only endpoints; ten passing HTTPS checks.
#12 Simulated notifications	Kamalpreet	Done	Loan/reminder/pickup records with unique event/channel protection.
#13 Branches, desks and transfers	Harsimran	Done	Staff location, dispatch/receipt and invalid-action guards; PR #6 and transfer evidence.
#14 Holds and FIFO queue	Kamalpreet	Done	Cancellation, repair allocation, destination readiness and protected collection.
#15 CSV book importer	Sandesh	Done	Separate copies, branch mapping and duplicate/header/invalid-row checks.
#16 Manager dashboard and exports	Sandesh	Done	Shared availability, fine audit and matching actual Windows CSV/PDF exports.
#17 Combined acceptance	Kamalpreet	Done	33 Windows tests, ten API checks and AUD 27 payment reconciliation.
#18 Installation and patron guides	Harsimran and Mehakpreet	Done	Common multi-branch setup and search/self-service instructions.
#19 Individual records and portfolio	All four; Kamalpreet compiles	Done	Named contributions, hours, reflections, minutes, diagrams and evidence.
#20 Separate VM verification	Harsimran	Blocked	Blocked: LocalDB Error 1920; SQL Server VSS Writer failed to start, including the administrator attempt.

Dependencies and integration

We integrated Sandesh's corrections through da0f483 into the tested 9b28459 lead checkpoint, which already included kiosk/API and branch/transfer contributions. A direct merge produced 18 conflicts. We resolved shared files around one checkout service and one migration chain rather than accepting competing versions indiscriminately.

We retained seven canonical migrations, explicit Identity-to-Borrower ownership, two-minute kiosk handling and durable simulated events. We kept InTransit=5 and a real Branch foreign key for pickup. We aligned Identity key lengths with the committed schema and checked pending model changes/duplicate table creation.

Catalogue, API, kiosk and reports use common eligibility. A physical Available copy can be reserved; the rightful Ready member can collect while later members wait. Manager outputs share one service, keeping recorded paid/unpaid fines separate from current estimates. CSV Quantity creates separate physical copies with unique codes and row-specific errors.

Allocations and hours

Member	Total/subtotal	Basis	Detailed record
Kamalpreet	30.76 hours	Retrospective estimate through 16 Oct 19:29.	Individual account/session record.
Harsimran	31 hours	Confirmed task hours; no daily start/end log supplied.	Individual ten-task table.
Mehakpreet	48 hours	Retrospective task estimate.	Individual nine-package table.
Sandesh	35 hours	Member-supplied retrospective task estimate.	Individual ten-task table.

The individual sections describe the work covered by these records. Assignment 2 baseline work is separate from the group project hours.

Risks and responses

Risk	Response/result
Shared model/migration conflicts	Agreed data contract, canonical history, model/script and real SQL checks.
Reserved/transit stock shown as available	Shared eligibility, transaction/index guards and traceable inventory rows.
Late integration	One combined release and targeted acceptance; less time remained for VM recovery.
Incomplete daily logs	Confirmed hours, retrospective estimates and partial sessions kept distinct.
VM LocalDB failure	Preserved troubleshooting/failure record; no successful alternative claimed.
Board maintenance	Updated the initial four issues to a 17-task board with owners, delivery statuses and a visible VM blocker.

Our strengths and weaknesses

We covered seven modules and checked a connected repair-to-pickup journey. Named branches and retained evidence helped trace contributions, and exact messages/export rows helped explain confusing results.

Our weaknesses were late integration, uneven confidence with Git/shared models, incomplete daily time logs and arranging a usable VM too late. We would set data contracts, trial installation and schedule smaller integrations near the start next time.

Rubric coverage

The supplied rubric totals 60: 37 group and 23 individual marks. The portfolio contains the criteria below.

Criterion	Marks	Evidence
Installation/running	5	Common guide, Windows/SQL pass; successful VM check still missing.
Team contracts	2	Four-member acceptance on original creation/sharing day; calendar date not retained.
Requirements	4	Seven modules, shared rules, decisions and acceptance matrix.
Team approach	4	Named ownership, shared contracts and tested integration.
Project plan	5	Milestones, dependencies, risks and a verified 17-task public board with owners and statuses.
Task allocations	10	Four named accounts; confirmed/estimated/session hours distinguished.
Meeting minutes	2	Discussion decisions, owners, actions and final technical status.
Strengths/weaknesses	8	Four first-person examples, learning and improvements.
Software requirements	10	All seven modules; Windows 33, API ten and actual browser/export evidence.
Source quality	5	Shared validation, transactions/indexes, migration checks, tests and audit.
Value adding	5	Each member explains practical contribution and connected journeys.

3. Requirements and decisions

We used the project brief to define the required scope. We expanded the existing .NET 10 MVC application through seven new modules/channels while retaining Books, Music and Toys. Our demonstration follows an unavailable toy through a fair queue, repair, branch transfer and collection. The decisions below connect those requirements to the implemented behaviour and acceptance evidence.

Users and access

User	Need	Access
Public patron	Search title/code/keyword and understand physical location/availability.	/Catalogue and /Kiosk
Linked library member	Scan, borrow, see only their own account and choose pickup.	/Kiosk/Scan, /Account and /Hold
Reception	Loans, requests, repairs and branch movement at a selected desk.	/Loans, /Holds, /StaffLocation, /Transfers
Admin	Maintain collection and bulk-import book copies.	Administration and /CsvImport
Manager	Borrowing, fine and inventory review/export.	/Manager and export actions
External developer	Protected read-only public catalogue projections.	/api/catalogue

Seven module requirements and acceptance

ID	Requirement	Owner	Acceptance evidence
FR01	Simplified kiosk with code scanning/typing, Identity-linked account, eligible 14-day checkout and private loans/requests. Reject inactive/unlinked membership and another member's allocation.	Mehakpreet; shared integration	Component checkout/account, PH01-PH03 and Windows rightful-owner SQL case. Fresh account request after Finish asks for login.
FR02	Secure GET-only catalogue, available copies, categories and operating status. Private API key, validated filters, no member account fields.	Mehakpreet	All ten HTTPS checks plus production middleware/filter tests.
FR03	Automatically simulate Email/SMS for borrowed, due-soon and overdue/fine events; record ready-for-pickup events without duplicates.	Kamalpreet	Historical loan/due-soon captures, C08 zero early notice and C11 destination messages, written recheck/restart results.
FR04	Active branches/desks and physical copy locations; dispatch at origin, receive at destination, prevent transit checkout and retain audit history.	Harsimran	Seven historical component captures and C09-C11 integration journey.
FR05	Holds on borrowed/damaged copies, FIFO by creation time/ID, one active member/item request and one allocation; notify only when ready at selected pickup.	Kamalpreet	H01-H03, C07-C13, PH01-PH03 and real SQL return/next-member cases.
FR06	Admin CSV utility with required headers and row validation; Quantity creates separate uniquely coded books at an active branch; repeated ISBN import creates none.	Sandesh	Production tests and actual 3-copy/2-row import, repeat 0/2 skipped, mixed 1/2 skipped and missing-year rejection.
FR07	Manager/Admin CSV/PDF borrowing, fine revenue audit and inventory health exports using dashboard definitions.	Sandesh; shared integration	Actual dashboard/CSV/PDF reconcile; AUD 27 payment transition verified.

Shared rules

Rule	Implementation and check
Physical Available is not sufficient for general borrowing.	Shared eligibility excludes active loan/transfer, usable reservation/eligible queue and inactive branch. Catalogue, kiosk, API and reports agree.
A rightful Ready member may collect while another waits.	CirculationService and Windows SQL ReadyOwner case.
No two active loans or allocations for one copy.	Per-item SQL transaction/lock and filtered unique indexes; simultaneous checkout creates one loan.
Pickup is a real branch; transit preserves previous enum values.	PickupBranchId references Branch.Id; InTransit=5 keeps values 1-4.
Inventory edit does not silently move a copy.	Create uses a valid branch; ordinary edits preserve it; transfers handle movement.
Repair away from pickup means Allocated, not Ready.	Destination receipt triggers readiness/messages; zero early notices are captured.
Current login determines private membership.	Explicit IdentityUserId, not a posted borrower ID or automatic email matching.
Date/fine definitions are consistent.	UTC event storage, Darwin calendar calculations, AUD 1.50 per overdue day, fixed-clock tests across midnight.
Recorded revenue and current estimates differ.	Returned-loan fines/paid state separate from active-loan estimate; actual payment leaves recorded total and estimate unchanged.
One compatible migration history.	Seven canonical migrations, no pending model changes/duplicate table creation; fresh SQL execution in isolated tests.

Inputs and quality requirements

Condition	Implementation/evidence
Admin-only import; Manager/Admin reports; Admin/Reception branch operations.	Roles and antiforgery metadata; selected signed-out/Patron denials. Forged-POST runtime checks remain unrun.
Shared kiosk privacy.	Finish/sign-out, no-store and two-minute idle handling. Normal idle and fresh-request logout confirmed; history-cache/server-only expiry not separately tested.
CSV at most 5 MB.	Title, Author, ISBN, Category, Quantity, PublicationYear required; Publisher optional. Quantity 1-100; year 1000 through earlier of 2100 and next year. Row results capped at 200 displayed with full counts.
API filters.	limit 1-100, category Book/Music/Toy, active branchId; search at most 100 characters. Invalid filter 400, missing item 404, POST 405, missing configured key 503.
Auditable exports.	Shared snapshot, AUD/ACST, full CSV inventory/fine rows; PDF first 200 fine/200 inventory rows with truncation note; formula-like CSV text escaped.
Same installation guide at each location, tested on VM.	Common guide provided; successful Windows/SQL run. VM LocalDB setup failed, so successful VM verification is incomplete.
Readable management/source access.	Public GitHub repository and board are readable without authentication; the task register records delivery and evidence.

Questions we resolved

Question	Our decision and reason
Which baseline?	Kamalpreet's Assignment 2 codebase, with one set of common models.
Can an email identify a borrower?	Use an explicit Identity link so coincidental email equality cannot choose account ownership.
When does off-site repair trigger pickup?	After destination receipt, so a patron is not sent to an unavailable copy.
Does Quantity mean one record with a count?	Create separately coded physical Book records; copies from the row may share an ISBN.
Is a current estimate fine revenue?	No; keep it separate until a return records a fine and payment is recorded.
What does operating status mean?	Service/database health and configured branch activity. We have no live opening-hours data.
How should competing shared files be merged?	Retain one service/migration chain, review compatibility and test the integrated behaviour.

Integration corrections addressed membership input, notification filter retention, rightful pickup collection, importer Quantity/year handling and manager availability definitions. We removed duplicate export buttons after inspecting the actual browser screen. Section 10 records the completed verification and its limits.

4. Discussion minutes

HIT339 Group 8 | Recorder: Kamalpreet, S389008

We compiled these minutes from our discussions and recorded follow-ups. Dates identify the follow-ups. Exact start/end times and complete attendance for the earlier conversations were not recorded.

M01 - Choosing the project and dividing the work

Dates: 3-4 October 2026

We chose Kamalpreet's Assignment 2 project as our shared starting point. We wanted the new modules to work together: a family could request an unavailable toy, keep its place during repair/transfer and collect at its chosen branch.

Responsibility	Owner	Next action
Holds and simulated messages	Kamalpreet	Define queue rules and coordinate requirements.
Branches/desks/transfers	Harsimran	Share the foundation before dependent modules.
Kiosk/API	Mehakpreet	Review login and borrower ownership before checkout.
Importer/manager exports	Sandesh	Review Book and report data structures.

We each needed to clone, build and run the project and retain our hours/tests/contribution evidence. Repository access was being arranged; Sandesh accepted the invitation on 4 October.

M02 - Schedule and dependencies

Dates: 10-12 October 2026

We set 15 October as our internal completion target, ahead of the official 16 October deadline. The first feature target was 13 October; on 11 October we requested working handoffs by 14 October so that integration/documentation could use 14-15 October.

The branch foundation was needed by holds, kiosk and imports. By 12 October it was running while transfers were still in progress. The patron login needed an explicit Borrower link.

Action	Owner
Share branch/desk models, item location, migrations and seeds.	Harsimran
Align Identity, checkout, hold and transfer guards.	Mehakpreet and Kamalpreet
Define branch mapping for imports and separate unpaid fines from collected revenue.	Sandesh

M03 - Bringing the branches together

Date: 13 October 2026

We discussed merge timing. Branch changes were already being brought together because other modules needed them. We also raised the risk of unfinished changes overwriting another contribution, especially in the shared layout, models, Program.cs and context/snapshot.

We agreed to review affected shared files with their owners, preserve each contribution and rebuild the combined application before accepting the integration.

Action: Kamalpreet to coordinate the combined review; module owners to identify their shared model/configuration dependencies.

M04 - VM requirement and locating Hold.cs

Dates: 13-14 October 2026

We checked that the installation guide must be tested on a VM through download, prerequisites, database setup and a working application for the selected branch. Harsimran owned the guide and VM check. Setup problems remained a blocker with no successful result.

Mehakpreet could not find Hold.cs on main. We clarified that it belonged to Kamalpreet's feature branch, and it was located on 14 October. Kiosk checkout needed the shared model rather than a second hold model. We also requested Sandesh's importer/report branch and delivery update that evening.

Actions: Harsimran to document/resolve the VM blocker; Mehakpreet to use the shared Hold model; Sandesh to supply the branch; Kamalpreet to coordinate integration.

M05 - Branch, hold and migration contract

Date: not recorded in the retained technical follow-up; these decisions were used in the 14 October integration.

We aligned the shared data before generating another migration:

Decision	Responsibility
Hold.PickupBranchId references Branch.Id.	Kamalpreet/Harsimran
InTransit uses value 5, preserving values 1-4.	Harsimran and module consumers
Checkout/API exclude transit and respect allocations.	Mehakpreet/Kamalpreet
Combine branch, hold, notification and patron Identity snapshots before migration.	Affected owners
Assign a valid branch on create, preserve it on ordinary edits and use transfers for moves.	Harsimran

BranchId had become required while the existing Book/Music/Toy inputs did not provide it. We needed the old create/edit workflows to remain compatible.

M06 - Final handoffs and corrections

Date: 15 October 2026

We restated the internal deadline and requested each member's task hours and own reflection, including a strength, difficulty, learning and improvement. Kamalpreet took responsibility for combining the portfolio and minutes.

We reviewed the importer/report handoff and kept its existing eight component cases. Before final acceptance we needed Quantity creating separate copies, valid publication years, hold/transit-aware availability, Darwin dates and loan-level fine details. We also needed a non-zero fine before/after payment rather than only zero-value summaries.

Action	Owner	Target
Correct import/report rules; supply samples, exports and final branch.	Sandesh	Internal handoff: 15 October
Resolve shared packages/layout/models while retaining working features.	Kamalpreet and owners	Before combined acceptance
Supply own task hours/reflection and identify estimates.	Every member	Final handoff
Compile portfolio, evidence and minutes.	Kamalpreet	Before submission

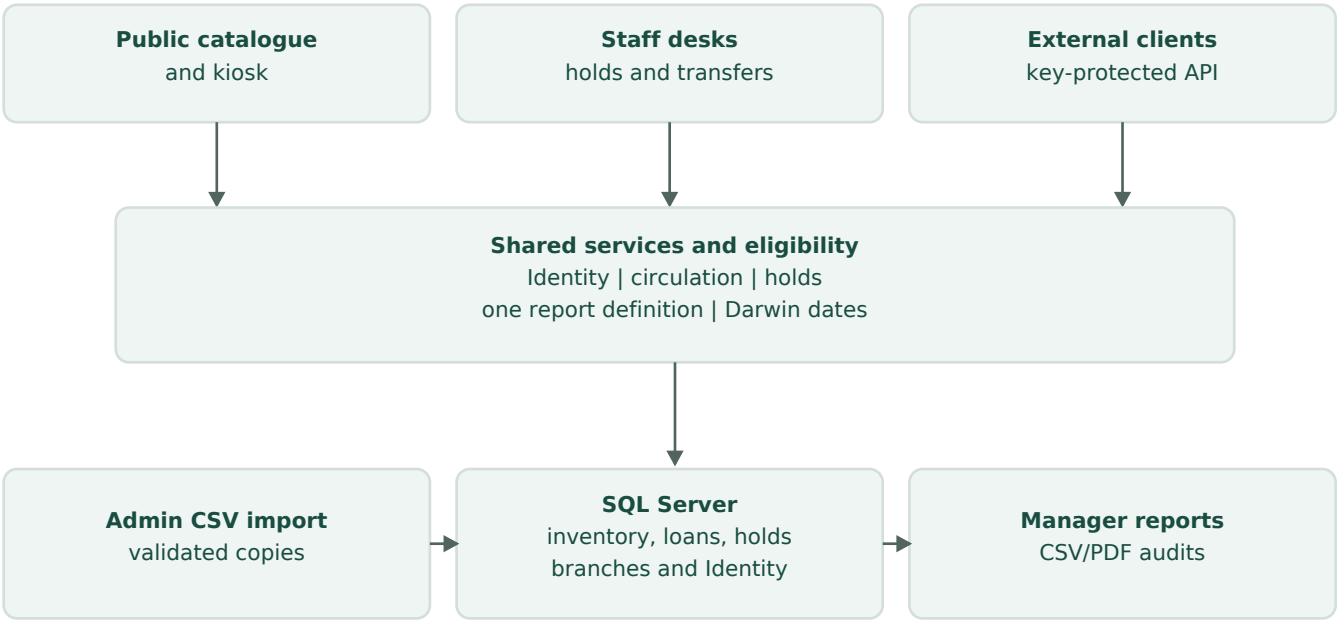
Action closeout - 16 October

This update records the final position of the agreed actions.

Action	Current position
Shared modules/schema	Combined release; seven canonical migrations retained.
Import/report corrections	Actual import captures, matching CSV/PDF and AUD 27 payment transition verified.
Combined tests	Windows build and 33 tests passed; all ten HTTPS API checks passed. Selected route denial and fresh-request logout matched expectations.
Individual handoffs	Four accounts received: Harsimran 31 confirmed hours; Kamalpreet 30.76, Mehakpreet 48 and Sandesh 35 estimated hours.
VM check	Blocked during LocalDB installation: SQL Server VSS Writer could not start.
Portfolio	Group records, four individual accounts, guides, diagrams and verification evidence compiled.
Project tracking	Seventeen assigned board tasks verified at 22:20 Darwin; all four foundation checks confirmed Done. VM blocker remains visible.

5. Integration and user journeys

Shared services and data



Simulated Email/SMS events are persisted with the shared data.

Figure 1. Browser channels, secure API and administrative import share one data model. Circulation, holds and reports use common eligibility; notifications are simulated durable records.

Shared concern	Choice	Purpose
Availability	CirculationRules.GenerallyAvailable and reservation queries.	Do not offer a promised/moving copy as general stock.
Checkout/return	CirculationService with per-item SQL transaction/lock.	Kiosk and staff protect the same loan, hold and transfer state.
Private ownership	Borrower.IdentityUserId from the authenticated login.	A posted member ID or coincidental email must not choose another account.
Queue/readiness	Shared placement/workflow services.	Keep FIFO and pickup location through repair and movement.
Event uniqueness	EventKey/Channel unique index and background scan.	A repeated check must not multiply the same event/channel.
Reports	One ManagerReportingService snapshot.	Dashboard/CSV/PDF need the same eligibility and fine definitions.
Schema	Seven canonical migrations and explicit Identity key lengths.	Avoid duplicate-table creation and model/schema drift.

Hold progression and physical movement

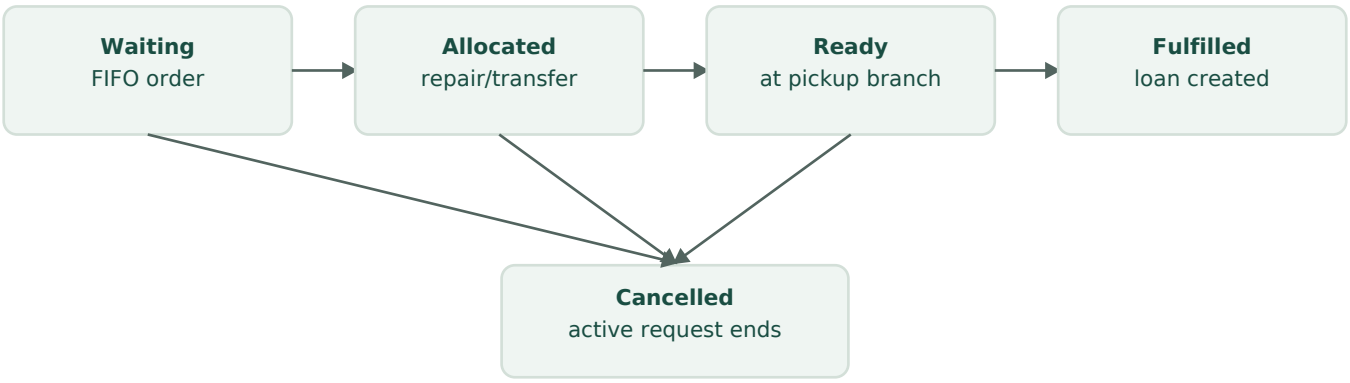


Figure 2. InTransit is an item/transfer state, not a HoldStatus. A request can stay Allocated while its copy travels. Destination receipt permits Ready; collection creates the loan and fulfils the request.

Journey A - repaired toy to family pickup

Stage	User action	System response	Evidence
Discover	Asha finds an unavailable toy and selects pickup at Palmerston.	Explain status and preserve the item code through login.	Catalogue/patron source and PH captures.
Join	Asha and Noah request the same borrowed/damaged copy.	One active request per member/item and stable FIFO.	H01-H03 and C07.
Prepare	Staff repair it at Darwin.	Allocate Asha, keep Noah waiting, send no early pickup notice.	C08 allocation/zero-record captures.
Move	Dispatch to Palmerston.	Prevent a different destination, origin receipt and transit checkout.	C09/C10 and transfer component tests.
Receive	PAL-D01 receives it.	Change physical branch, make Asha Ready and log two pickup channels.	C11 receipt/readiness/messages.
Collect	Asha checks out at the pickup branch.	Fulfil her request and create one active loan while Noah waits.	C12 and real SQL rightful-owner test.
Reuse	Return a usable copy.	Record any fine and promote the next eligible request.	C12 next-ready and real SQL return test.

Journey B - an auditable management decision

An administrator selects an active branch and imports a CSV. Quantity=2 creates two separate codes. Repeating existing ISBN rows skips them. A manager can then see each copy's branch, physical condition, reservation restriction and general availability in the inventory audit.

A current overdue estimate is separate from a recorded fine. When a returned loan's AUD 27 fine is recorded as paid, total recorded fines stay 27, paid moves from 0 to 27 and unpaid from 27 to 0. The payment timestamp is retained and the current estimate stays 9. Each figure can be traced to the loan that caused it.

6. Individual work, hours and reflections

Each member describes their contribution, working hours, strengths, difficulties and learning. Retrospective estimates are labelled in the hours tables. The original reports and source history support these accounts.

6.1 Kamalpreet - S389008

Student number: S389008 | Role: team lead, holds, simulated notifications and integration

My contribution

We used my Assignment 2 application as the foundation. I worked on holds and simulated notifications and coordinated shared circulation, branch and patron changes in Visual Studio. My aim was to keep a family's request connected to the condition and physical location of the item, so that repair or transfer would not lose its place or send an early pickup message.

My allocation	Delivered outcome	Evidence
Requirements/coordination	Scope, roles, dependencies and shared data decisions.	Requirements, group record and minutes.
Holds and queue	One active request per member/item, FIFO, cancellation and one allocation per copy.	H01-H03 and C07-C13.
Patron placement	Private Identity-linked request/progress; inactive/unlinked guards.	PH01-PH03 and shared queue captures.
Simulated notifications	Loan, reminder, fine and pickup events with duplicate protection.	Notification records and Email/SMS captures.
Shared workflow	Repair allocation, destination readiness, protected collection and next-member promotion.	Integrated journey and real SQL tests.
Acceptance/evidence	Guided Windows checks, exact outcomes, screenshots and export reconciliation.	33-test run, ten API checks and actual CSV/PDF files.
Interface/portfolio	Green theme, clearer navigation, evidence captions and final documents.	Views/theme source and portfolio.

My earlier checkpoints include 74668e1, dbbc540, a1e8190, 70db5b7 and 9b28459. On 16 October I imported the combined b79faf3 release. The final documentation and duplicate-export-button cleanup follow that tested calculation/circulation source. Each member's history remains attributed in the repository.

My testing

I ran the Windows build and Test-Release.ps1 -IncludeSqlServer: 33 passed, zero failed and zero skipped. The three real SQL cases cover rightful pickup collection, simultaneous checkout and late-return/queue handling. All ten HTTPS API checks also passed.

I imported three physical copies from two CSV rows and repeated the file with zero new copies. I checked mixed invalid rows and a missing header and supplied the actual manager CSV/PDF. Both exports agreed on nine items, four generally available and AUD 27 recorded fines. After I recorded the

fine as paid, paid became 27 and unpaid zero, while total fines, the separate current estimate and inventory stayed unchanged.

I confirmed signed-out and Patron route restrictions and that a fresh My library account request after Finish asks for login. Earlier inactive/unlinked account captures show the membership guards. I kept captured outcomes separate from my written results and unrun cases.

My personal reflection

Strengths in practical testing

One strength I showed was continuing to investigate when a result did not make sense. A membership number appeared in capital letters on the form but was rejected because I had entered lowercase letters. I reported the difference between what I typed and what the screen displayed, then applied the validation change. This was a useful reminder that a form can look correct while still creating a problem for the person using it.

The repair and transfer journey helped me understand why testing one screen is not enough. Asha's toy was damaged at Darwin and her chosen pickup branch was Palmerston. Repair allowed allocation, but the pickup message had to wait until the destination received the copy. I followed that journey through collection, the next patron's turn and another return requiring repair. Checking those changes together made the purpose of each status clearer to me.

Working through unfamiliar integration

My main weakness was limited confidence with Git and Entity Framework integration. I needed guidance to understand automatically appearing merge changes, resolve shared files and decide what to commit. I also found it difficult to keep the documentation current while testing. Asking for help allowed me to continue, but I would reduce this dependence next time by learning the branch and migration workflow before the team begins changing shared models.

Lessons from testing and coordination

I sometimes found similar looking test steps confusing. One example was a notification screen showing six rows that included both loan confirmations and pickup messages. I had to apply the pickup filter before comparing the count. I learned to state the item, message type and expected state before testing, then report the exact result. A short completion reply alone does not explain every part of a test.

I would agree the shared model and service boundaries earlier, bring the branches together in smaller batches and record each result while the relevant screen is open. I would also keep a dated time log from the beginning. Coordinating four modules made me more aware that having code on separate branches and having one tested release are different stages of the work.

My value added

My contribution keeps the pickup request meaningful through repair and physical movement. The patron hears that the copy is ready only when it has arrived at the chosen branch. Asha and Noah's journey made this benefit concrete.

The private hold form lets an active linked member join the same queue that staff manage and follow their own progress. The simulated notification history helps explain an unexpected message count. I also helped bring the separate modules into one release.

My hours and improvement

My **estimated effort is 30.76 hours** through 16 October 19:29 Darwin. It covers coordination, development, integration, testing, evidence and documentation. My working-session summary follows.

Next time I would agree data contracts earlier, integrate smaller changes, keep a dated task/time log and test the VM guide near the beginning. I would learn the Git/migration workflow before the team changes shared models. The VM installation problem showed me why setup verification needs its own early milestone.

My working-session details

Kamalpreet | S389008 | Record through 16 October 2026, 19:29 Darwin

My **estimated effort is 30.76 hours**. The retrospective record covers the activity periods below.

Date, 2026	Work covered	Estimated session time
03 Oct, morning	Brief and locating my Assignment 2 baseline	0 h 42 min
03 Oct, afternoon	Build, repository and project-board setup	1 h 49 min
04 Oct	Roles and completion plan	0 h 17 min
11 Oct, daytime	Holds, messages and application checks	4 h 02 min
11 Oct, evening	README, interface and shared Identity/module coordination	2 h 19 min
14 Oct, morning	Rubric and remaining-work review	0 h 31 min
14-15 Oct, overnight	Merge, repair/transfer/collection tests, evidence and patches	8 h 18 min
15 Oct, daytime	Build/package fix, patron tests, member reports, wording, minutes and VM coordination	10 h 49 min
16 Oct, evening	Release import, 33 Windows tests, CSV/PDF/payment audit, API and access/logout checks	1 h 58 min

Total estimated effort: 30.76 hours (approximately 30 h 46 min). Session figures are rounded to minutes. This record covers recoverable project activity through the stated cutoff.

6.2 Harsimran Singh - S388232

Student ID: S388232. Responsibilities: branches, reception desks, staff working location, interbranch transfers, installation guide and VM installation attempt.

Delivered work

I implemented the Branch and ReceptionDesk models, unique codes, branch migration and repeat-safe seed data, staff working-location selection, item location assignment and transfer dispatch/receipt. Items remain associated with their origin branch during transit and move to their destination on receipt. Ordinary inventory edits preserve physical location; changes between branches use the transfer workflow.

My report supplied on 16 October contains seven preserved screenshots from 13 October: dispatch, wrong-branch rejection, destination receipt, database confirmation of Palmerston location, checkout rejection during transit, repeat-receipt rejection and Manager access denial. These are historical component results. The integrated held-item journey is recorded separately in the circulation evidence.

Confirmed working hours

My confirmed task total is **31 hours**, allocated as follows.

Task	Confirmed hours
Branch and reception-desk models	2
Database migration and seed data	3
Working-location selection and display	3
Transfer dispatch and receipt	5
Checkout and transfer validation	3
Book, Music and Toy compatibility changes	3
Functional testing and evidence	3
Installation guide and report	3
VM setup and troubleshooting	4
Integration review and compile checks	2
Total	31

I include the VM troubleshooting because I performed it even though setup failed. I count this time once.

Strengths and weaknesses

The strengths of my contribution include implementing a clear dispatch/receipt workflow and invalid-action checks alongside successful transfers. I also ensured my work was visible through feature-branch commits and screenshots.

A weakness was arranging an appropriate VM late. Installation issues prevented successful verification of the guide. Earlier coordination of database and hold-service changes would have provided more time for integration testing.

Learning

This task helped me understand how a shared model change affects existing application features. The required branch relationship also needed inventory creation/edit location preservation. I learned to retain existing data during migrations, validate staff locations on the server and maintain transfer history.

Reviewing the integration helped me understand why checkout, reservations and transfers share locks and availability rules. I learned to distinguish implemented behaviour, code review and tests supported by actual evidence.

Value added

My contribution lets staff track item movement across branches, including origin, destination, dispatch time, receipt time and status. The working-location display identifies the selected branch and reception desk. Validation messages explain wrong-branch receipt and checkout during transit. Inventory compatibility changes assign new items a valid branch and preserve existing locations during ordinary edits.

My installation result

I did not achieve a successful VM installation. I installed Git and .NET on the Darwin VM, but SQL Server 2022 LocalDB failed when SQL Server VSS Writer could not start. Trying with administrator permissions also failed; I have not established the cause. Another environment restricted installation permissions. The combined Windows/SQL tests passed separately and do not replace the blocked VM check.

My original 12-page report retains seven component captures from 13 October and the VM attempt. The confirmed 31-hour table above is my current record.

6.3 Mehakpreet Kaur - S390303

Student ID: S390303. Responsibilities: Self-Service Kiosk and Secure Read-Only Catalogue API.

Personal reflection

Working on the TopEndLibraryHub project was a valuable learning experience that allowed me to develop my technical knowledge, problem-solving skills and confidence in software development. My main responsibilities were developing the Self-Service Kiosk and Secure Read-Only Catalogue API. These components required me to work with the existing ASP.NET Core MVC application, SQL Server database, Entity Framework Core, authentication system and GitHub repository.

At the beginning of the project, one of my biggest challenges was understanding how the existing application was structured and how my assigned components would interact with the work completed by other team members. Rather than developing a completely separate application, I needed to ensure that my kiosk and API could operate within the shared library management system. This helped me understand the importance of reviewing existing code, following project requirements and considering integration from the beginning.

My first major contribution was developing the self-service kiosk interface. I worked on creating a user-friendly layout that allowed library patrons to search the catalogue, filter items by category and availability, and enter library codes to identify items. I focused on making the interface simple and accessible because kiosk users should be able to complete basic library activities without needing extensive technical knowledge. Developing these features improved my understanding of Razor views, MVC controllers and the relationship between front-end interfaces and database records.

The most challenging part of my contribution was implementing the checkout functionality. Initially, although the kiosk interface and item scanning worked, checkout did not complete successfully. Through testing and troubleshooting, I identified that the authenticated user account needed to be securely linked to a library borrower record. I worked on integrating the IdentityUserId relationship so that the system could identify the correct library membership without relying only on matching email addresses. After resolving this issue using a dedicated test membership, I successfully demonstrated the checkout of Discovering the Top End (BK-0001), including a 14-day borrowing period and a due date of 28 October 2026. Seeing this loan subsequently displayed in the correct patron's account was one of the most satisfying moments of the project because it confirmed that several separate components were communicating successfully.

Another important responsibility was developing the secure catalogue API. I worked on read-only access to library catalogue information and API-key authentication. During testing, I confirmed that a request without an API key was rejected with an appropriate error message. This helped me appreciate the importance of protecting application endpoints, validating requests and distinguishing public interfaces from secured application services.

Database integration was another significant learning experience. I worked with Entity Framework Core models, relationships, migrations and SQL Server records. I also needed to consider borrowing restrictions involving item availability, active loans, holds, branches and transfers. These requirements helped me understand that a successful checkout must involve more than changing an item's status; it must respect the rules of the wider library system.

I experienced several technical difficulties during development, including compilation errors, migration inconsistencies, merge conflicts and problems launching the application through HTTPS. Resolving these issues required patience, careful investigation and repeated rebuilding and testing. In particular, Git conflicts involving ApplicationDbContext.cs, Item.cs, Program.cs and the Entity Framework model snapshot taught me how easily shared files can be affected when several developers work on the same project. I learned the importance of preserving existing functionality,

checking duplicate properties and reviewing merged code rather than automatically accepting one version.

Collaboration also played an important role in my development. My kiosk depended on other team members' work involving holds, notifications, branches and transfers. This encouraged me to communicate about shared borrowing rules and understand the responsibilities of different modules. I managed my changes through a dedicated Git feature branch, committed and pushed my work, and created Pull Request #8. Successfully merging my contribution into the group's main branch was an important achievement because it demonstrated practical experience with collaborative version control.

Testing was essential throughout the project. I tested the kiosk welcome page, catalogue searching, item scanning, successful checkout, patron account information, logout and API-key restrictions. I also prepared screenshot evidence to support my contribution. Alongside the technical implementation, I produced an API testing report, kiosk integration documentation, user-feedback documentation and a patron user guide. Preparing these documents helped me understand that software development involves not only writing functional code but also communicating how the system operates and how it should be maintained and used.

If I were to complete a similar project again, I would establish shared database conventions and integration requirements earlier, coordinate changes to common files more closely, and test merged functionality more frequently. I would also maintain a more detailed development log from the beginning so that my time spent coding, debugging, testing and documenting could be recorded accurately. These improvements would help reduce integration problems and make collaboration more efficient.

Overall, this project strengthened my practical skills in ASP.NET Core MVC, C#, Entity Framework Core, SQL Server, authentication, API development, testing and GitHub collaboration. More importantly, it taught me how to approach unfamiliar technical challenges systematically, take responsibility for my assigned work and persist when a solution does not work initially. I am proud of completing my kiosk and API contribution and successfully integrating it into the group repository. The experience has increased my confidence in contributing to larger software development projects and highlighted the importance of teamwork, testing, security and continuous learning.

My task hours

Mehakpreet Kaur | S390303 | Retrospective task estimate

Task	Estimated hours
Requirements, existing code review and interface planning	5
Kiosk welcome, catalogue search, filters and scanning	8
Identity integration and borrower linking	5
Checkout, borrowing restrictions and duplicate protection	6
Patron account summary and session privacy	4
Catalogue API and API-key security	5
Shared models, migrations, rebuilding and merge troubleshooting	7
Kiosk/API tests and user-feedback record	4
Technical/patron documentation and Git collaboration	4
Total	48

6.4 Sandesh Bhattarai - S390535

Student number: S390535 | CSV book importer and manager dashboard/CSV/PDF exports

My contribution

I adapted the importer and exports to the shared Book, Item, Branch and loan models. My branch is feature/sandesh-csv-reports and my pull request is #7. My initial implementation is 2ab17fa, code corrections run through da0f483 and final documentation is 97e53b6. The integration retains these histories and records later shared-rule corrections separately.

My reflection

Contribution and application improvement

The primary thing I did was the CSV book importer and the CSV/PDF exporter for the manager dashboard. The importer enables an administrator to add many book records from a file instead of entering them one by one. It checks for required fields, stops duplicate ISBN records, produces unique library codes, and attaches imported books to a branch. I have put links in the interface so that it's easy to find these tools. The manager downloads make it easy to evaluate and maintain as records, borrowing and fine summary numbers.

A strength, with an example

When I was troubleshooting unknown code, I was persistent. The importer was originally written by me in my previous solo project, however the group application used BookGenre instead of Genre, and required both a properly formatted LibraryCode and a BranchId. I checked out the Book, Item, Branch and database context classes, fixed the importer, then showed I could successfully import 100 books.

A difficulty, response and outcome

The most difficult component was adapting my work to a broader programme that was being updated by other people in the organisation. I had troubles with the .NET SDK, missing package references, build errors. I went through the bugs one by one; I tried the features in the browser and checked the branch assignment with a read only Local DB query for book BK-0102. The record was associated with Darwin Hub (BranchId 1).

What I learned and what I would improve

I learned more about ASP.NET Core MVC, Entity Framework Core, CSV validation, QuestPDF reporting, and git feature branches. Next time I would review group project models before coding, create smaller and earlier changes, track my actual time daily and design negative and integration tests from the beginning. I would also work with the group on reporting definitions before considering the FinePaid indication as a payment audit.

My strengths, weakness and value

My strength was persisting through unfamiliar model/package errors and checking the actual source and browser/SQL results. My weakness was bringing the combined work together late. The earlier component captures could not establish corrected Quantity/year behaviour or a non-zero fine audit, so the final release needed focused integration checks.

My importer reduces repeated manual entry by creating separately coded physical books from a CSV row. My manager exports make borrowing, fine and inventory information portable. The combined report explains why a physical Available copy can still be reserved and separates recorded payments from current late-charge estimates.

My hours

My estimated effort is **35 hours**, allocated as follows.

Task	Estimated hours
Requirements review and coordination	2
CSV importer development	5
Quantity, ISBN and year improvements	3
Manager dashboard/calculations	4
CSV/PDF exports	4
Database/migration troubleshooting	4
Manual functional/access tests	4
Automated xUnit examples	3
Git/PR/conflict resolution	3
Screenshots and report	3
Total	35

My evidence and learning

My final report has ten captures and reported cases covering bulk/repeat/header import, branch assignment, restricted access and a transfer scenario. One historical image shows Year 0; current validation rejects that value. My eight xUnit examples are component arithmetic/status checks, separate from the combined 33 production/SQL tests.

The later actual Windows import/report/payment results are recorded separately. I would review the common models earlier, integrate smaller changes, agree reporting definitions first and keep daily actual hours next time.

My source history

Sandesh Bhattarai | S390535 | feature/sandesh-csv-reports | PR #7

The following six commits are retained in our integration history. Git author metadata identifies source checkpoints, not working duration.

Commit	Author date/time	Change
97e53b6	16 Oct 18:36:46 +09:30	Final individual documentation
da0f483	16 Oct 00:06:22 +09:30	Fine/payment-status test examples
bb23ecf	15 Oct 23:02:34 +09:30	Duplicate branch/transfer migration correction
6f5f0e8	15 Oct 21:51:03 +09:30	Duplicate BranchId correction
0aaf002	15 Oct 21:14:02 +09:30	Quantity/year and reporting correction
2ab17fa	15 Oct 00:35:00 +09:30	Initial importer and CSV/PDF exports

My branch also includes master merge 2725467. The later integration adds shared-rule reporting and production-code tests separately. My updated original report is retained in docs/member-reports/Sandesh_Testing_Report_16Oct.pdf.

7. UX feedback and refinements

Mehakpreet collected informal patron feedback from Anjana, Robinpreet and **Navdeep**. They requested no specific further changes. Our internal testing also identified the input and workflow observations below.

Observation	Change or response	Result/limit
Lowercase membership input looked uppercase but failed validation.	Case-tolerant MBR regex with uppercase storage.	Source/regex checks verified; separate lowercase browser retest not recorded.
Damaged-copy checkout's generic message confused the test instruction.	Matched the expected result to the actual Borrow item screenshot.	Rejection visible; no damage-specific message change claimed.
Mixed notification types made pickup counts unclear.	Explicit item/type/channel filters and retention on immediate recheck.	Filtered total four confirmed; redirect regression passed.
Manager page had duplicate export-button pairs.	Removed the redundant pair in the final source.	Presentation-only cleanup; report calculations unchanged.
Shared kiosk needs clear next steps/privacy.	Collection, Self service, Join waiting list, My account and Finish, with Identity ownership.	Own-account/inactive/unlinked captures and fresh-request logout; no formal accessibility study claimed.

Our green theme supports the practical flow. The main benefit is explaining repair, allocation, transit and readiness without treating an origin allocation as ready at the destination. We did not measure staff time saved or comparative novelty.

We use the green library theme and direct task names such as Collection, Self service, Join waiting list and My library account. The interface explains why a copy cannot be borrowed and what happens next. The informal reviewers requested no further changes; the internal input/filter/button observations above led to concrete improvements. We do not present these as a formal accessibility or measured usability study.

8. Installation and running guide

These instructions install the combined .NET 10 MVC application for each physical branch. The Windows-tested revision is b79faf3. The separate VM attempt failed during LocalDB installation; its result is recorded below.

Prerequisites

Use a supported Windows environment capable of running the .NET 10 SDK and SQL Server Express LocalDB, such as a suitable Windows 11 VM. Install Git, .NET 10 SDK, SQL Server Express LocalDB and a current browser. Visual Studio is optional; if used, it must support the installed SDK. The integration build was checked on Linux with SDK 10.0.402/runtime 10.0.13 and EF Core 10.0.10. LocalDB execution requires the Windows environment and is a separate check.

Verify `git --version`, `dotnet --info` and `sqllocaldb info`. Start the LocalDB instance if needed using `sqllocaldb start MSSQLLocalDB`. Check VM free disk/RAM before installing; retain VM/OS versions and the actual commands/results in the installation record.

Obtain the final source

```
git clone https://github.com/Kamal03102006/HIT339_Group8_TopEndLibraryHub.git
cd HIT339_Group8_TopEndLibraryHub
git switch integration/group8-release-16oct
git rev-parse HEAD
dotnet restore .\TopEndLibraryHub.csproj
dotnet dev-certs https --trust
```

The source branch is integration/group8-release-16oct. The project is `TopEndLibraryHub.csproj`; the solution is `TopEndLibraryHub.slnx`. The commit printed by git rev-parse identifies the installed revision.

Select the database and configuration

For a fresh installation choose a new demonstration database through user-secrets; SQL Server creates it when startup migrations are applied:

```
$connection = 'Server=(localdb)\MSSQLLocalDB;Database=HIT339_Group8_Demo;'
$connection += 'Trusted_Connection=True;MultipleActiveResultSets=true;'
$connection += 'TrustServerCertificate=True'
dotnet user-secrets set "ConnectionStrings:DefaultConnection" $connection
$apiKey = [Guid]::NewGuid().ToString('N') + [Guid]::NewGuid().ToString('N')
dotnet user-secrets set "CatalogueApi:ApiKey" $apiKey
dotnet user-secrets set "Kiosk:BranchCode" "DAR"
dotnet build .\TopEndLibraryHub.csproj
dotnet run --launch-profile https
```

Keep the API secret private and do not paste it into committed files or evidence captures. Use the HTTPS URL printed by the application (the recorded profile uses port 7197). Startup applies committed migrations and creates repeat-safe demonstration data, roles, staff accounts and three branch/desks. Development also creates the separate linked Sky patron account described in README.

An existing lead database is compatible only when it follows this canonical history:

Order	Migration
1	00000000000000_CreateIdentitySchema
2	20260906021222_AddLibraryDomainModels

Order	Migration
3	20261011030009_AddHoldsAndNotifications
4	20261011090000_LinkPatronIdentity
5	20261011133133_AddBranchesAndReceptionDesks
6	20261012101617_AddItemTransfers
7	20261014074832_LinkHoldsToPickupBranches

The earlier consolidated `20261014104445_IntegrateHoldsBranchesTransfers` belongs to a different historical branch and is not added to this chain. If that history is present, retain the old database and choose a separate fresh demonstration database for this release. Do not delete user data, rewrite migration history or generate another migration to hide a duplicate-table error. The new report/import/service changes require no new schema migration.

In Visual Studio, `Get-Migration -Context ApplicationDbContext` shows the history; `Update-Database -Context ApplicationDbContext` can apply pending canonical migrations before launch. Startup does the same application step automatically. The offline model/script test checks for pending model differences and repeated creation of shared tables; it does not execute SQL Server migrations.

Run at each physical location

Sign in with the local demonstration Admin or Reception account. Open Working location, select the branch and a matching active desk, and confirm the location displayed in the navigation. DAR/DAR-D01, PAL/PAL-D01 and CAS/CAS-D01 use identical software/setup. New inventory uses the staff's validated working branch. Ordinary edits preserve existing location; transfers move an item through dispatch and destination receipt.

A physical kiosk uses the trusted server configuration `Kiosk:BranchCode`; change it to PAL or CAS for a station at those branches, then restart. A patron cannot select a different checkout branch through form input. Use separate patron accounts and profiles for private-account testing. A new registration is not automatically linked to an existing borrower by email; use the controlled fictional linking process in Documentation/Kiosk_Integration_Readme.md.

Local demonstration accounts

Role	Email	Password
Admin	admin@topendlibrary.local	Admin#2026!
Reception	reception@topendlibrary.local	Reception#2026!
Manager	manager@topendlibrary.local	Manager#2026!
Development patron Sky/MBR-8001	patron@topendlibrary.local	Patron#2026!

These fictional accounts are for the local assessment. The linked patron is seeded only in Development and has no staff role.

Verify installation and restart

On the VM, follow the guide from source download through migration/seed and launch. Record OS/VM, Git/.NET/LocalDB versions, final commit, configuration method, actual result and corrections. Verify login, location selection at each branch, catalogue, kiosk account, API with a local key, a safe fictional circulation/transfer journey, importer and manager exports. Stop/restart the app; check that previous demonstration records are retained and branches/desks are not duplicated. Keep screenshots or logs of meaningful outcomes. Run the supplied Windows tests and record their actual results separately.

Known VM blocker and troubleshooting

During our VM attempt we installed Git and .NET in the Darwin VM, but SQL Server 2022 LocalDB installation failed with Error 1920: SQL Server VSS Writer could not start. An administrator attempt also failed, and Event Viewer reported a startup timeout. Another VM/environment blocked installation through administrator restrictions. The underlying cause is not confirmed. These are failed attempts, not installation passes.

For build-file locks in a OneDrive project, stop the app and Visual Studio, then run dotnet build-server shutdown. Remove only generated bin/obj output before rebuilding. Retain source, screenshots, migrations and database files.

Repeatable release checks

Stop the app before building. From the repository root:

```
.\scripts\Test-Release.ps1 -IncludeSqlServer
dotnet run --launch-profile https
```

In a second PowerShell window in the same folder:

```
.\scripts\Test-CatalogueApi.ps1 -BaseUrl 'https://localhost:7197'
```

The API script prompts for the private local key without displaying it. Clients send X-Api-Key. Use the actual URL printed by the application if different. SQL tests create unique TopEndLibraryHub_Tests_* databases and clean up only those; they do not reset application data. Our Windows run passed all 33 tests; the separate VM installation result is recorded above.

Our actual installation result

The combined Windows build and all 33 SQL-enabled tests passed at b79faf3. This verifies the application and database operations in that environment. The required separate VM installation did not reach a working application: LocalDB failed when SQL Server VSS Writer could not start, and the administrator attempt failed too. The cause of the installer failure was not established.

We also resolved a locked generated AssemblyInfo.cs file in the OneDrive project by stopping app/build servers, removing generated bin/obj and rebuilding. Updating Pkcs 9.0.4 to 9.0.21 removed the supplied build warnings; the package audit reported no vulnerable packages from its configured sources at that check. We would trial the VM guide early and check resources/permissions before the final integration period next time.

9. Patron search and self-service guide

We provide this guide for the combined release. Browsing is public; private account/borrowing uses a verified login linked to membership.

1. Open the application and select **Collection** or **Self service**. Browsing does not require login.

2. Search by title, code or keyword. Choose a category and availability filter; use Clear filters to start again.

3. Read the location and availability. **On hold** means a usable copy is reserved; **In transit** means it has not arrived. Borrowed, damaged or destroyed copies cannot be borrowed.

4. For a borrowed or damaged copy, select **Join waiting list** on its catalogue card. Sign in with your verified linked library account, check the item code and select your pickup branch. Submit once. **My library account** confirms the request and shows your waiting position. Your place is preserved while staff handle return, repair and transfer. Ask Reception for help if your login is unlinked or your membership is inactive.

5. Select **My library account** and sign in using your verified linked account. You can see your own current loans, dates, recorded unpaid fines, separately labelled estimated late charges and active pickup requests. An unlinked login needs staff assistance.

6. At the item's physical pickup branch, select **Scan an item**, scan its barcode or type the library code, then choose **Borrow this item** when offered. The account shows the new loan and due date. An item reserved for somebody else cannot be borrowed.

7. Ask staff for help if the item is at another branch, in transit, awaiting repair or your membership is inactive. A catalogue listing alone does not guarantee checkout.

8. Select **Finish and sign out** before leaving a shared kiosk. Signed-in kiosk screens, including the public browsing screen, have a two-minute inactivity reset.

Kiosk checkout defaults to a 14-day loan. Estimated late charges use AUD $1.50 per overdue Darwin calendar day; staff confirm the final fine when the item is returned. Members can place requests and view their own progress. Staff can place assisted requests and cancel them. One member can have one active request per item. Joining a waiting list does not mean an item is ready for collection; wait for **Ready for collection** at your chosen branch. This assessment application records simulated email/SMS messages.

For a local Development demonstration only, use the fictional linked patron in the integration handoff. Do not use another person's card number to access their account.

Understanding your request

Message	Meaning	Next action
Available	No general borrowing restriction identified.	Check the physical branch and use kiosk/staff checkout.
On hold	The copy has a reservation or eligible request.	Do not assume it is available to everyone.
Waiting / Awaiting repair or return	The request is queued and the copy is not ready.	Keep the request and follow progress.
Allocated	A copy is assigned but may still need to travel.	Wait for readiness at the chosen branch.
In transit	Dispatched but not yet received.	Wait for receipt; borrowing is blocked.
Ready for collection	The reserved copy is at the selected pickup branch.	Sign in there and collect your own allocation.
Inactive / not linked	This login cannot create a loan or request.	Ask authorised staff to check membership status/link.

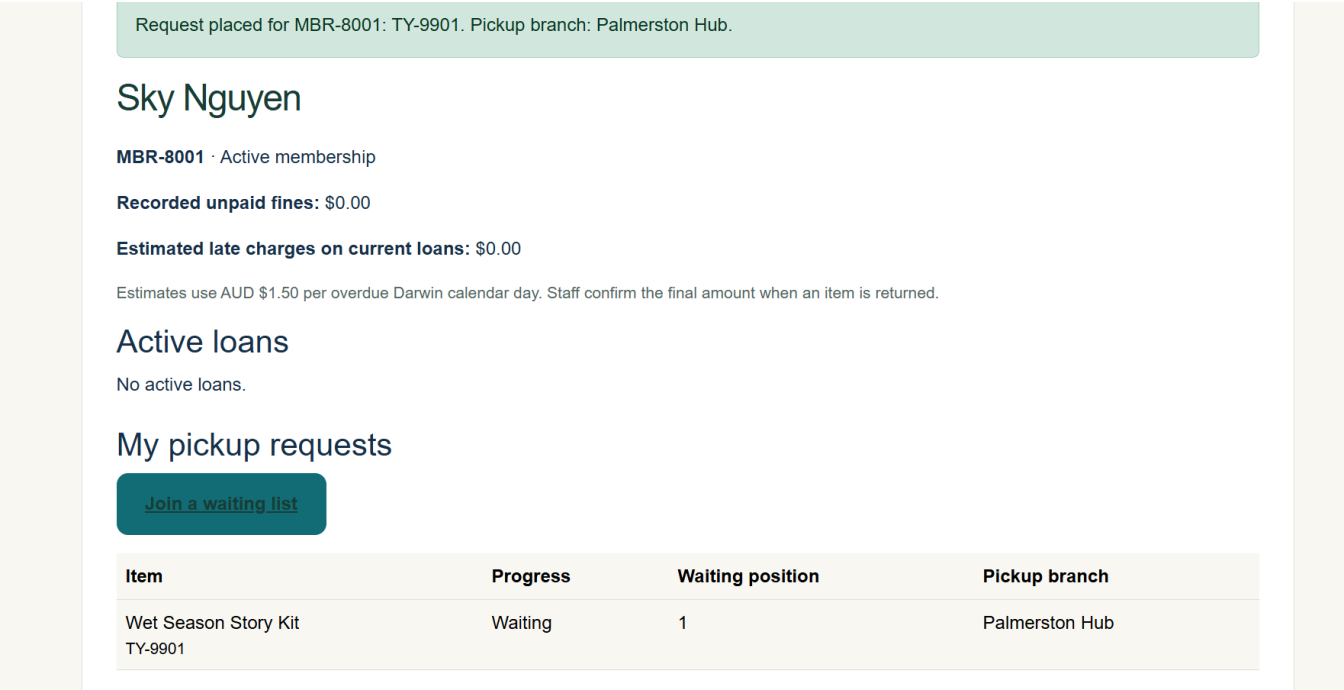


Figure 3. Sky's account confirms their own request at Palmerston. This 15 October capture shows Waiting, not readiness or collection.

10. Verification and source quality

Our verification combines Windows browser and SQL checks, Linux fixtures and the retained component evidence.

Check	Result	Evidence basis
Linux build/audit	Build passed; no vulnerable packages reported at the check.	Retained logs; SDK 10.0.402/runtime 10.0.13, EF 10.0.10.
Linux suite	30 passed, 0 failed, 3 SQL skips.	Retained TRX/text. InMemory does not test SQL locking/foreign keys.
Windows build/SQL-enabled suite	33 passed, 0 failed, 0 skipped at b79faf3.	Kamalpreet's Windows terminal transcript; TRX retained on the test machine.
Canonical migrations and circulation	Fresh isolated migration execution, rightful-owner collection, simultaneous checkout and late return passed.	Three real SQL cases.
Actual HTTPS API	All ten checks passed; key not printed.	Script passing output and explicit all-ten confirmation.
CSV browser	3 copies/2 rows, repeat zero; mixed 1 accepted/2 skips; missing year header rejected.	Actual captures and inventory.
Manager CSV/PDF	Nine-item dataset reconciles to dashboard and audit rows.	Actual exported files inspected.
Non-zero payment	AUD 27 moves unpaid to paid; total/estimate/inventory unchanged.	Actual before/after CSVs.
Browser access/Finish	Signed-out challenge, Patron denial and fresh-request login after Finish.	Written Windows outcomes.

Production tests cover fine/payment calculations, shared report eligibility, import validation/quantity, CSV formula escaping, PDF generation, API key/filters, role/antiforgery metadata and notification filter retention. A fixed clock crosses Darwin midnight. Synthetic exports use 13 items and AUD 9 fines; they are labelled separately from the actual nine-item Windows dataset and older 104/107-item member captures.

The separate VM installation failed at LocalDB setup. Runtime forged-POST, browser-history-cache and JavaScript-disabled expiry checks were outside the completed checks. Normal kiosk idle behaviour was observed; the source and metadata checks cover the implemented controls.

Detailed test results, transcripts, captures and actual export files are indexed in docs/test-evidence.

Test depth

Our fixtures call production FinePolicy, payment, importer, reporting/export, API-key and filter code. Cases cover invalid quantities/years, separate physical copies, duplicate imports, reserved/transit stock, CSV formula-like text, report totals and repeat-payment timestamps. A fixed clock crosses Darwin midnight. A separate model/script check finds no pending model changes and checks the seven migration names and repeated shared-table creation.

EF InMemory does not establish SQL translation, foreign keys, transactions or locking. The real SQL cases create isolated databases and apply the canonical migrations before checking rightful Ready collection with a waiting follower, two simultaneous checkouts producing one loan and late return recording an AUD 6 fine while promoting the next request. All three ran in the 33-test Windows suite.

The synthetic report uses 13 fictional items, three generally available, three reserved Available and two in transit; AUD 9 recorded fines split 3 paid/6 unpaid, plus a separate 1.50 estimate. The actual Windows dataset below has nine items. We keep these and older member captures with 104/107 items separate.

Actual Windows import and report audit

I ran the Windows browser checks on the b79faf3 release and retained the captures and exported files. The audit compared the dashboard, parsed CSV rows and both pages of the PDF export.

Import	Actual result
books-valid.csv	3 physical copies / 2 accepted rows / 0 skipped.
Repeat same file	0 copies / 0 accepted / 2 skipped.
Mixed validation sample	1 accepted row / 1 copy / 2 skipped; BK-0005 Another Valid Book visible. Lower error rows are outside the capture.
Missing year header	Missing columns: PublicationYear; no Missing Year Column item in actual inventory.

The dashboard and before-payment CSV/PDF agree: 9 items, 4 generally available, 1 reserved Available, 0 in transit, 2 borrowed, 2 damaged, 0 destroyed; 7 loans, 2 active, 5 returned and 2 overdue. Before CSV generated 18:25:44 ACST; PDF 18:25:53. The nine-second generation-time difference is expected.

Loan 3 / TY-0001 / MBR-0001 was due 26 September and returned 14 October: 18 days at AUD 1.50 = AUD 27.00. The after CSV generated 18:37:46 records payment at 18:37:23.

Metric, AUD	Before	After
Recorded fines	27.00	27.00
Recorded paid	0.00	27.00
Recorded unpaid	27.00	0.00
Separate current late-charge estimate	9.00	9.00

All inventory audit rows and loan/item counts stay unchanged. BK-0002 through BK-0005 are generally available. TY-9001 is physically Available but reserved; damaged TY-9901 with a pending request is unavailable but not a reserved Available copy. Both PDF pages were readable without observed clipping. A duplicate export-button pair in the older capture was removed from the final source without changing calculations. A repeat payment is tested automatically, not inferred as another browser run.

API contract

Tester: Kamalpreet, S389008. I used a fresh private local key and ran Test-CatalogueApi.ps1 against https://localhost:7197. All ten checks passed; the script kept the key private.

Case	HTTP result
Missing key / wrong key	401 / 401
Catalogue / available / categories / status	200 / 200 / 200 / 200
Invalid limit / invalid category	400 / 400
Missing item	404
Read-only POST	405

The script also checks selected private-field names in successful responses. This is regression coverage, not a full penetration test. Status means health/configured branch activity, not live opening hours.

Private access and its limits

Tester: Kamalpreet, S389008. The table records the selected Windows browser outcomes.

Check	Actual outcome
Signed-out /Manager and /CsvImport	Login required.
Patron /Manager and /CsvImport	Access denied.
Private account	Initially matched expectations; a later unlinked-account warning left that browser/account context uncertain. Earlier Sky captures remain separate ownership evidence.
Finish, then fresh /Kiosk/Account request	Asked for login, confirmed at 19:08 Darwin.

The logout result is based on a fresh request that required login. Forged POST, browser-history-cache and JavaScript-disabled expiry were outside these browser checks.

11. Selected evidence

The selected figures document queue order, cancellation, allocation, destination readiness, import validation and reporting. The repository evidence index links each capture to its test and retains complementary images where a result spans two screens.

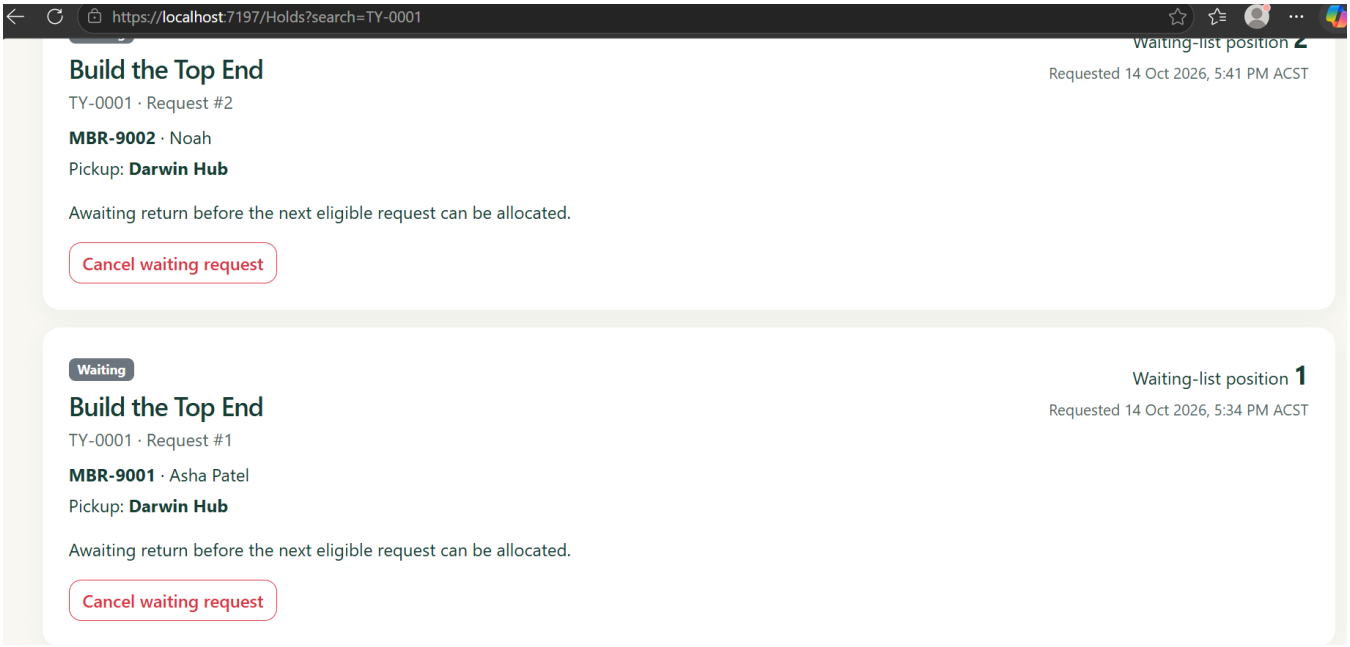


Figure 4. Asha is first and Noah second for borrowed TY-0001. The complementary overview is retained in the repository.

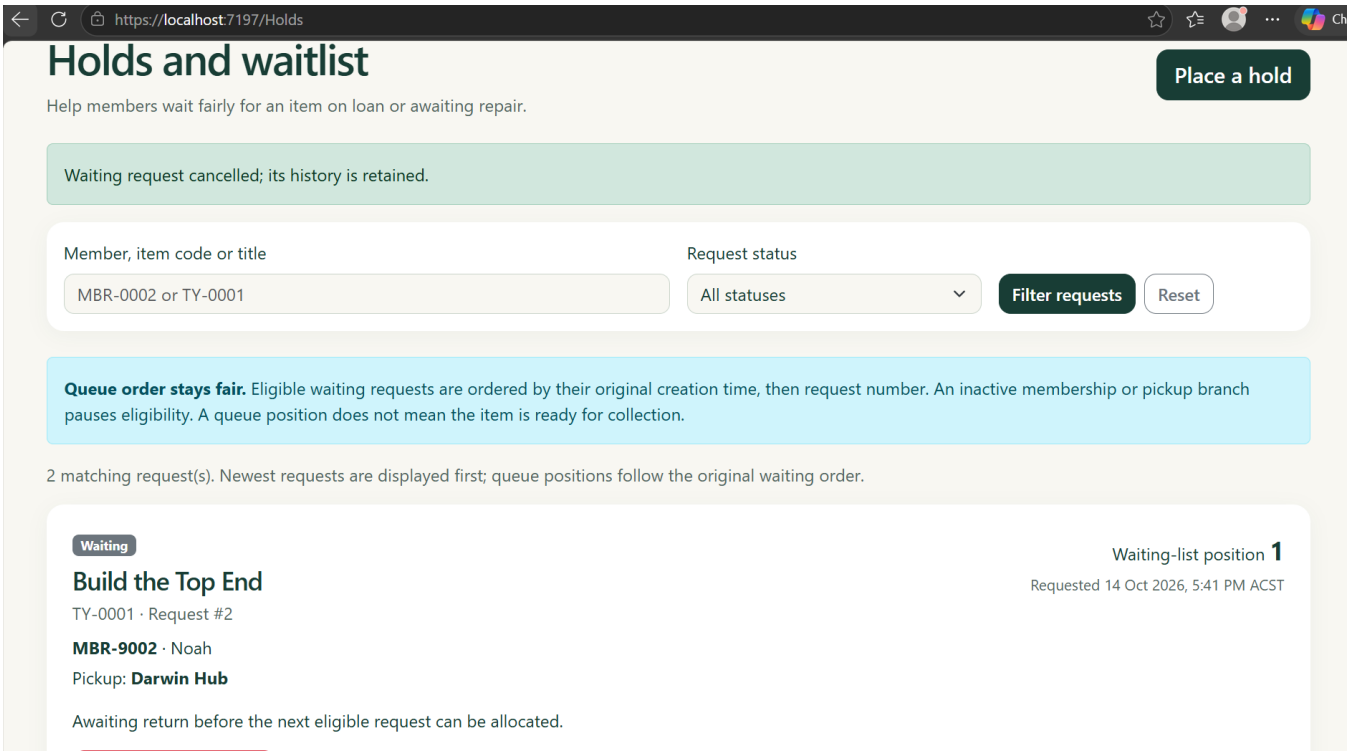


Figure 5. Cancellation promotes Noah; the complementary history capture retains Asha's Cancelled record.

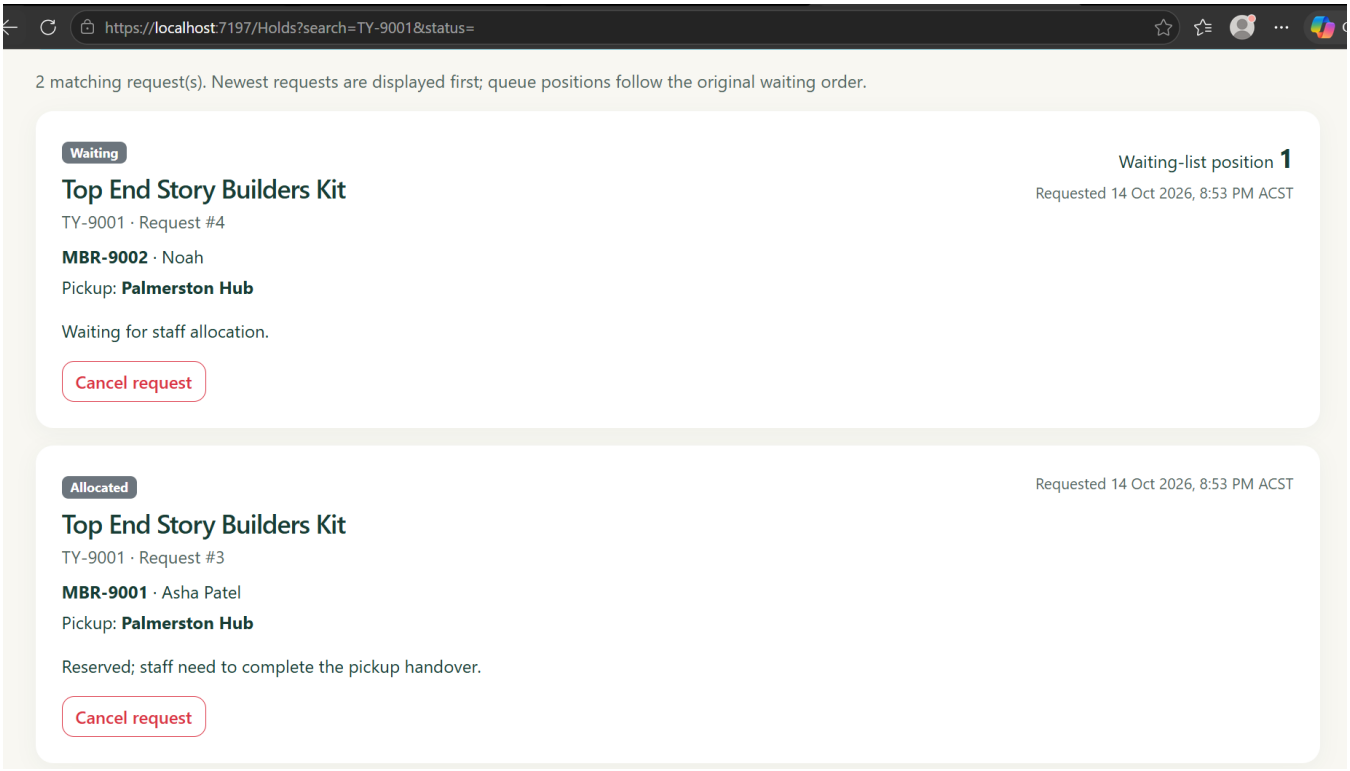


Figure 6. Repair allocates Asha for Palmerston while Noah waits. Allocation at origin does not mean readiness at destination.

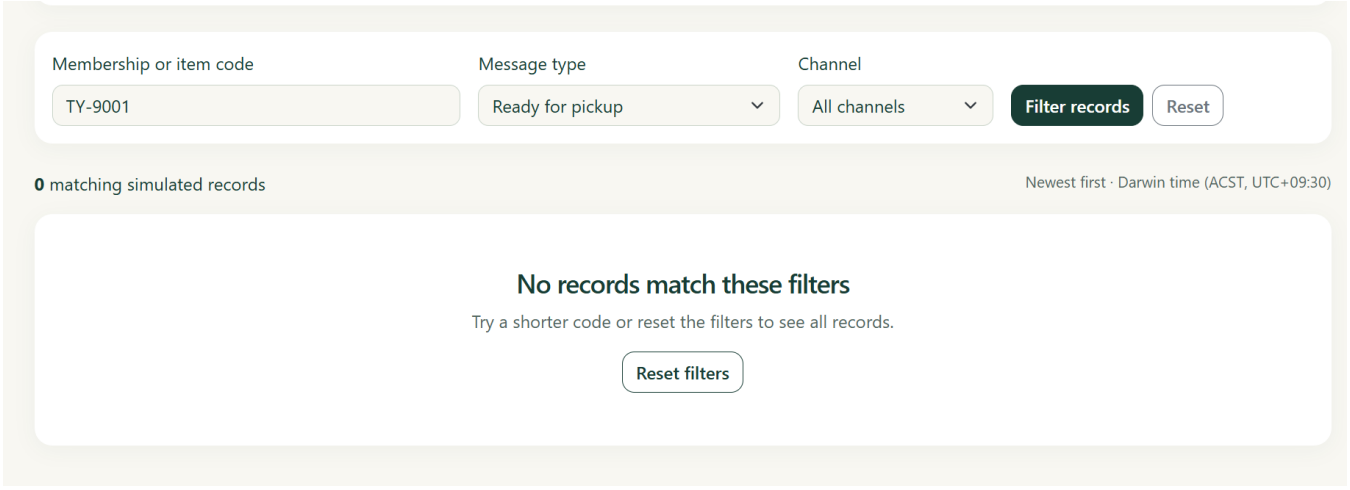


Figure 7. The filtered TY-9001 pickup history has zero records before arrival. The messages are simulated Email/SMS records.

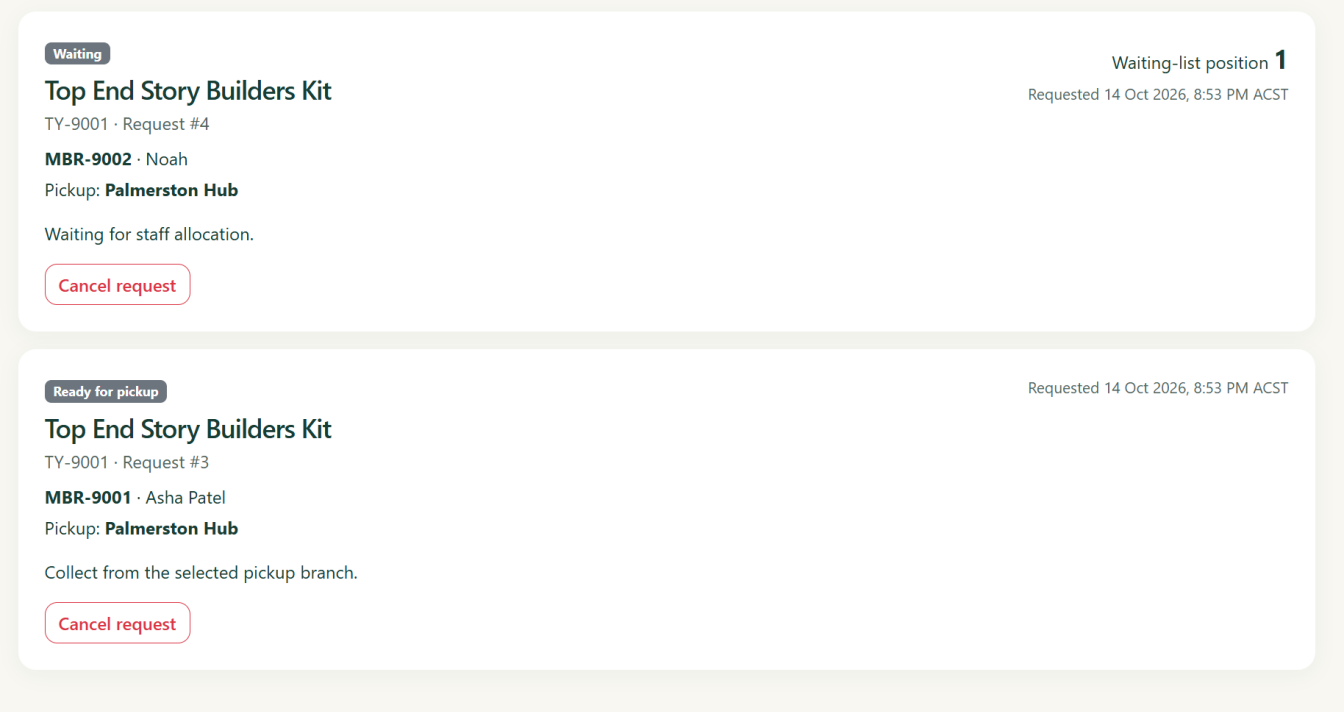


Figure 8. Destination receipt makes Asha Ready while Noah still waits. The receipt and two channel messages are separately indexed.

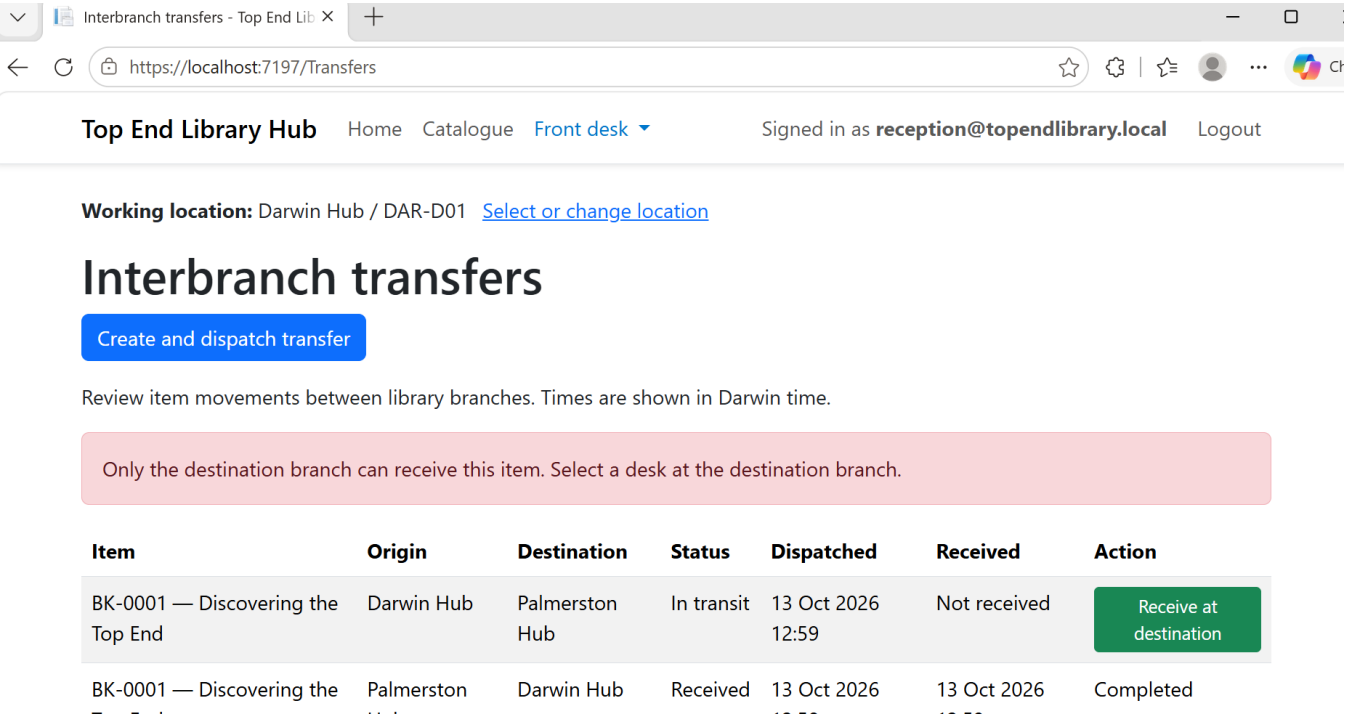


Figure 9. The 13 October component check rejects receipt at the wrong branch.

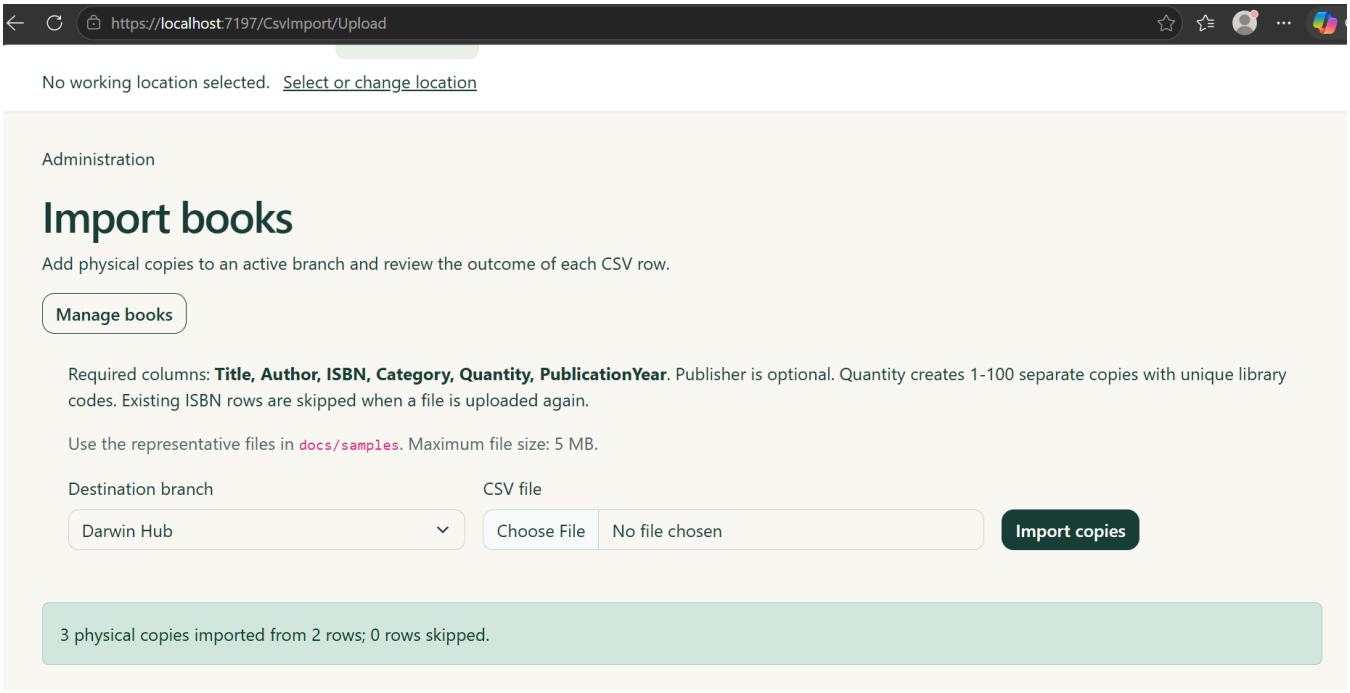


Figure 10. Actual Windows import: three physical copies, two accepted rows, zero skips. Repeating the sample created none and skipped two ISBN rows.

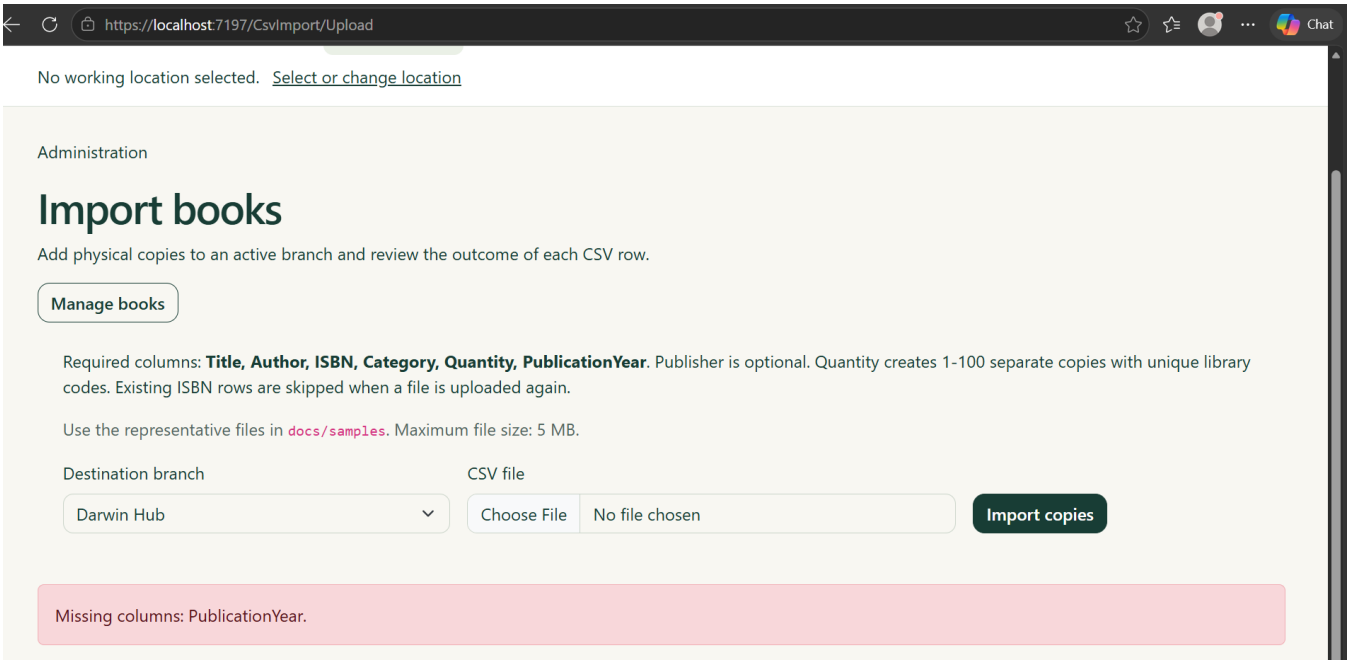


Figure 11. The actual importer rejects missing PublicationYear. The mixed sample/capture and accepted inventory are retained separately.

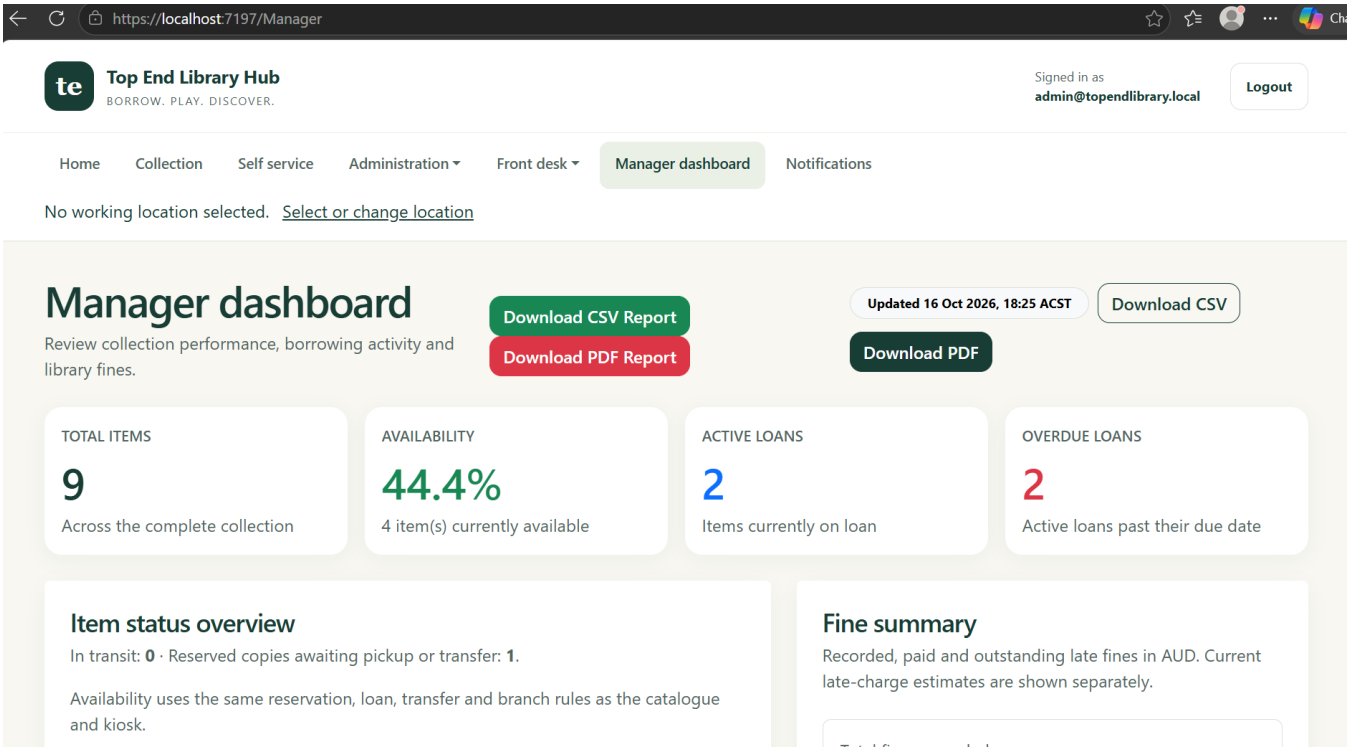


Figure 12. Nine items, four generally available, two active/overdue loans and one reserved Available copy. Fine totals below this crop are verified from actual exports. The duplicate button pair visible here was removed in the final source.

12. Evaluation, sources and acknowledgement

Evaluation and limitations

The combined release supports the seven modules and the connected repair-to-pickup journey. Shared circulation rules keep availability consistent across the catalogue, kiosk, API and manager outputs. The actual Windows tests and exported audit rows provide evidence for those behaviours.

The separate VM installation failed during LocalDB setup. External Email/SMS delivery and bank payment processing are outside the implementation: notifications are simulated records, and fine payment is recorded by library staff. The browser security checks cover the specific scenarios listed in section 10.

Earlier shared data agreements, smaller integration batches and an early VM trial would have reduced the final integration pressure. Updating the board after each handoff and keeping daily task/time logs would have made progress easier to track throughout development.

Source index

Source	Purpose
Group Project.pdf	Supplied two-page brief: Parts A/B, VM check and PDF/repository deliverables.
Supplied rubric	Eleven criteria and weights totalling 60.
Rohan's assignment-two announcement	Requirements questions, marketing, feedback acted on, breadth and depth as design prompts.
Public Git repository and project board	Source history, tasks and marking read access.
docs/REQUIREMENTS.md	Seven-module requirements, acceptance checks and shared business rules.
docs/PROJECT_TASK_REGISTER.md	Full task ownership, completion status and evidence at the integration review.
docs/MEETING_MINUTES.md	Discussion decisions, responsibilities and action closeout.
docs/individual/	Four personal accounts, hours and verified source history.
docs/member-reports/ and Documentation/	Original member reports and kiosk/API/patron material.
docs/screenshots/README.md	Evidence captions and original filename mapping.
docs/test-evidence/	Linux logs/TRX, Windows records, actual exports, payment/API results and labelled synthetic outputs.

My acknowledgement of assistance

Kamalpreet, S389008

I used ChatGPT/Codex to explain Git and Entity Framework, help draft and review code, plan test cases, organise evidence, and draft documentation and my reflection. It also helped organise my retrospective time record.

I applied changes in Visual Studio and supplied my Windows outcomes and screenshots. On 16 October I used Codex to prepare the combined source, correct import/report rules, write production-code tests, run the Linux build and fixtures, and organise the release. I then ran the Windows build and all 33 tests and supplied the actual browser exports, payment result and ten-check HTTPS API pass.

This statement describes my own use of assistance. Each member's contribution is identified in their individual account and source history.